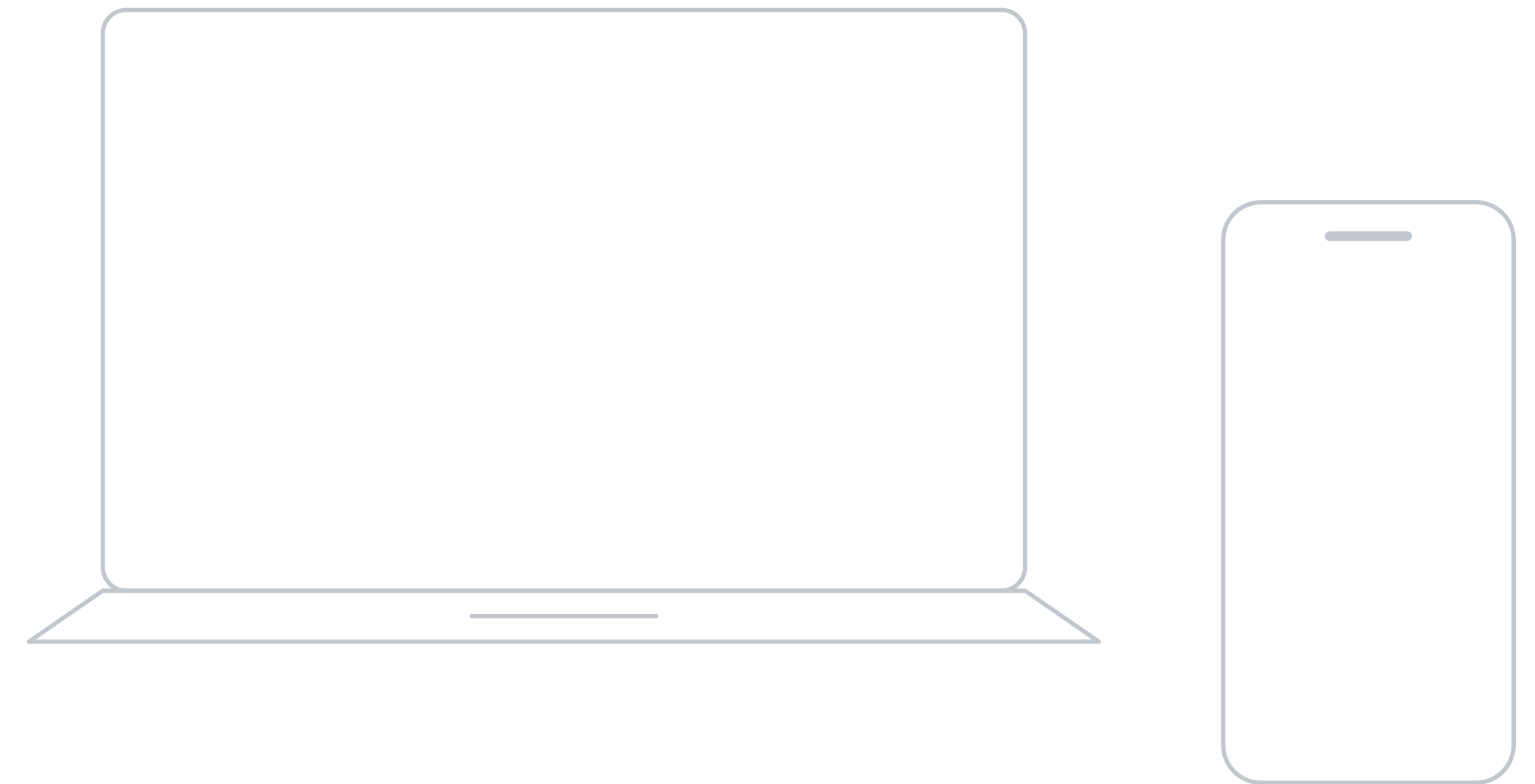


One model. Many behaviors.

Local AI is bespoke AI.



The same model should be able to

-  write code
-  read contracts
-  handle finance
-  translate
-  summarize
-  teach

...

and let you choose which ones apply.

Select the ones you want.
Set how strongly each applies.

Two things you give a model

Knowledge

what it knows

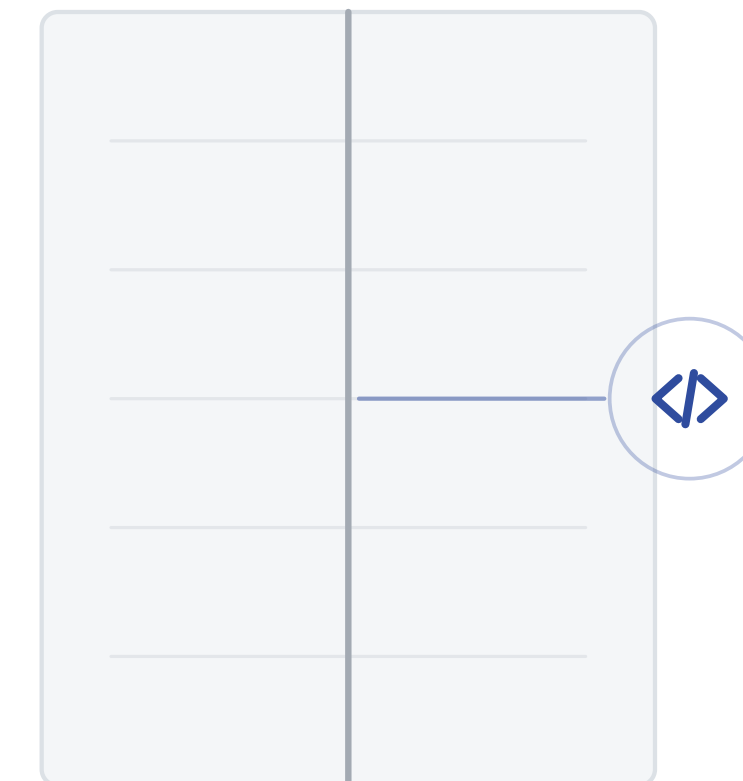
YOU SUPPLY THIS TODAY



Behavior

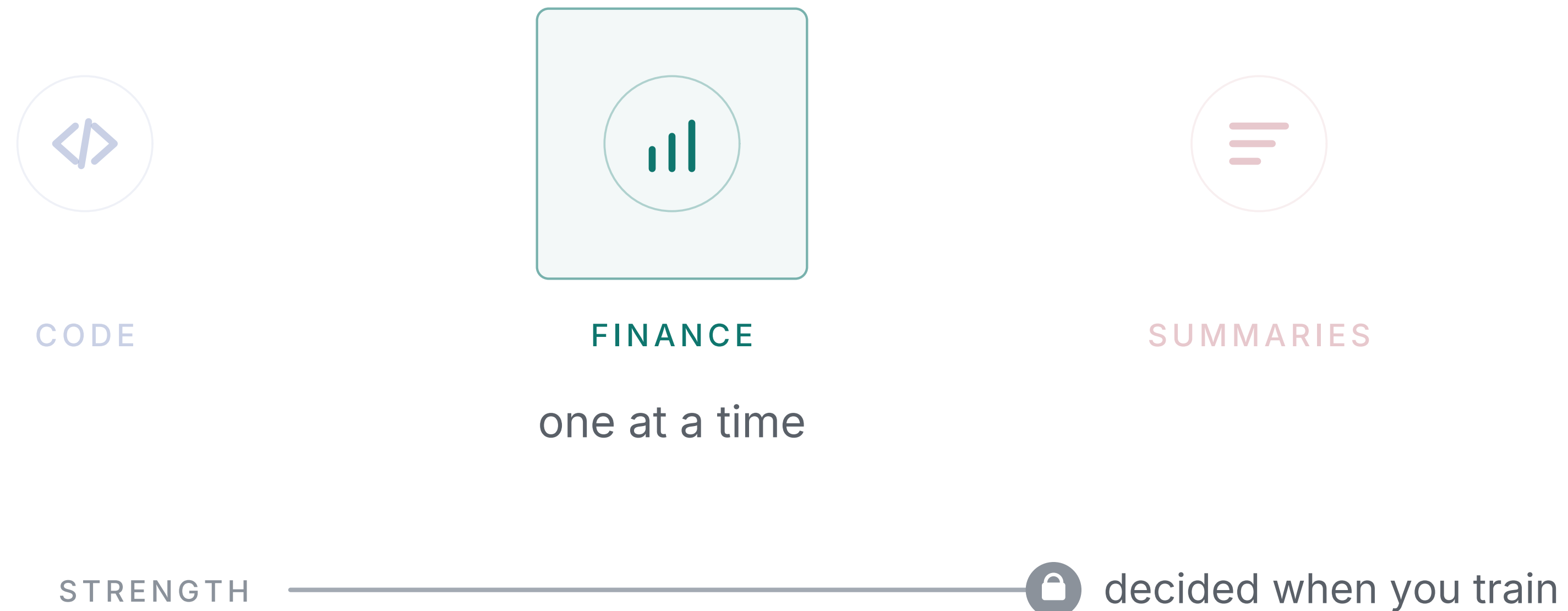
how it responds

YOU CAN SUPPLY THIS TOO



Context made knowledge something you supply.
Behavior can work the same way.

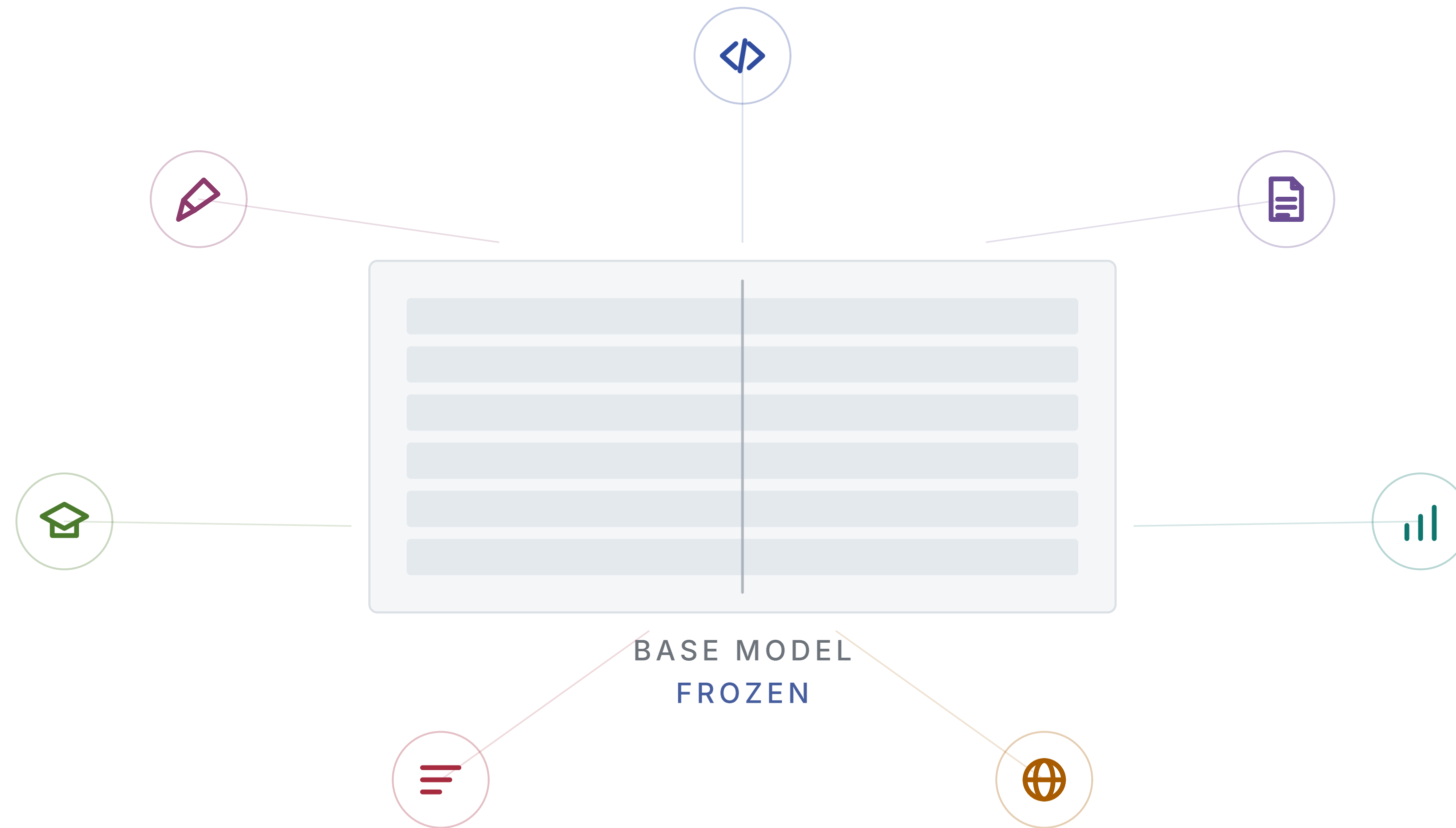
How adaptation works today



A behavior is selected, not continuously composed.
Its learned effect is fixed when you train.

What if it could be?

A different arrangement



Every behavior stays resident on one frozen model.

LARA

Lightweight Additive Residual Adaptation

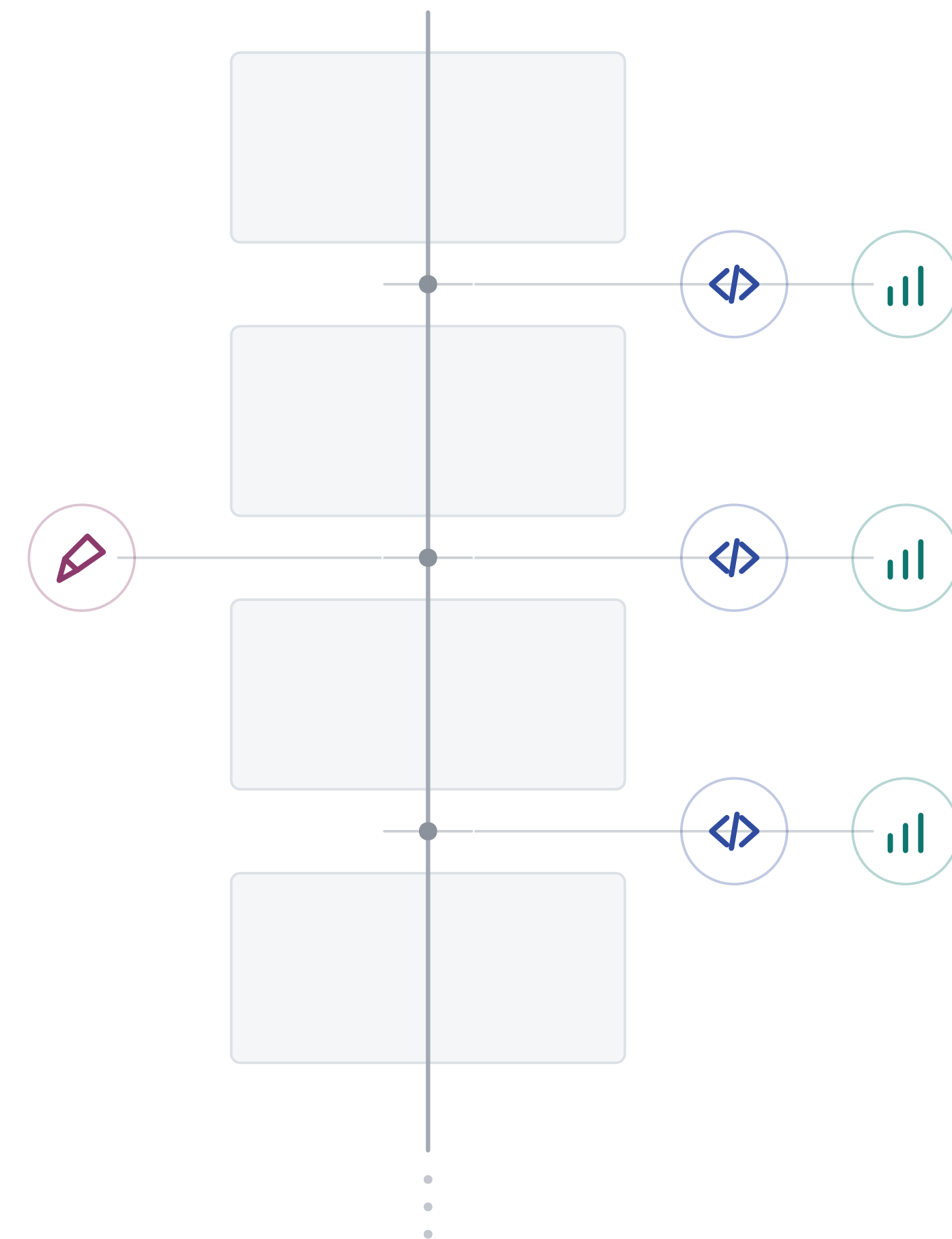
It reads the hidden state between layers, computes a low-rank correction, and adds it back to the stream.

The weights are never written to.

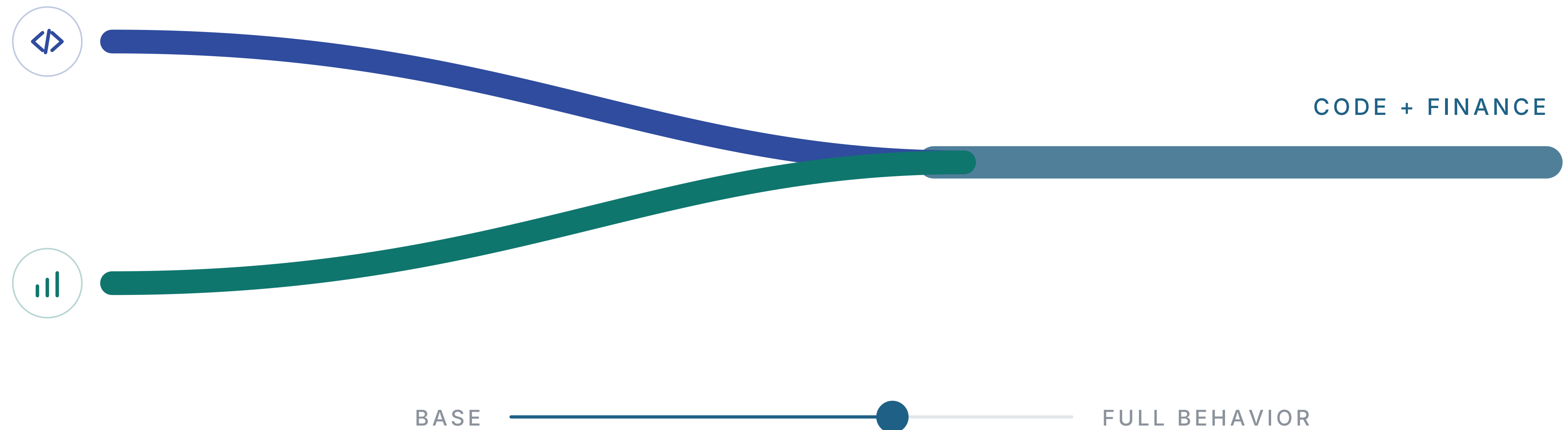
Matches LoRA at equal parameters, on fine-tuning and on preference optimization.

$$h \leftarrow h + \gamma \cdot (\alpha/r) \cdot \text{up}(\text{down}(\text{LayerNorm}(h)))$$

RESIDUAL STREAM

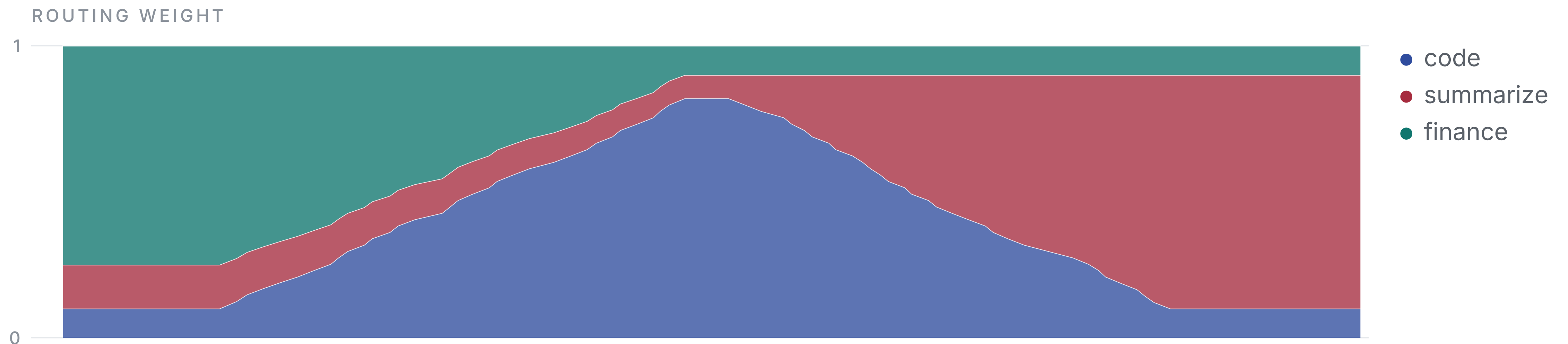


Behaviors combine



Strength is a value you set at runtime.
The same behavior can apply lightly or fully.

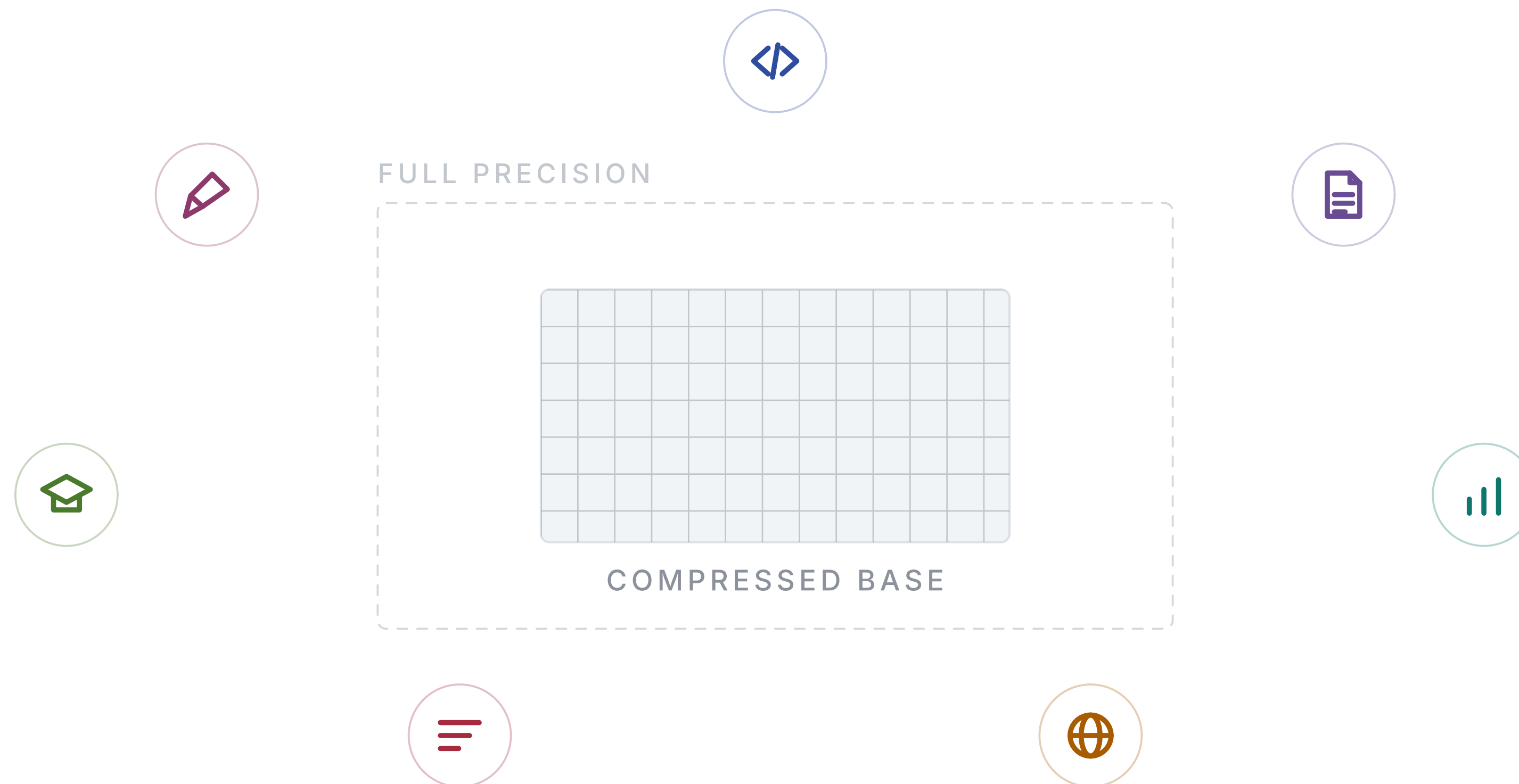
Routing, per token



The margin dropped to 4.2%, so compute $\text{delta} = (\text{new} - \text{old}) / \text{old}$ and summarize the result plainly.

The mix moves as the text is generated.
No discrete switch between them.

It runs on quantized models



The correction never touches the weights,
so how they are stored does not matter.

EVERY RESULT IN THE PAPER WAS PRODUCED ON AN 8-BIT BASE

One model. Many behaviors.

A library you install from.

